7 Building Custom Object Detectors

This chapter delves deeper into the concept of object detection, which is one of the most common challenges in computer vision. Having come this far in the book, you are perhaps wondering when you will be able to put computer vision into practice on the streets. Do you dream of building a system to detect cars and people? Well, you are not too far from your goal, actually.We have already looked at some specific cases of object detection and recognition in previous chapters. We focused on upright, frontal human faces in *Chapter 5*, *Detecting and Recognizing Faces*, and on objects with corner-like or blob-like features in *Chapter 6*, *Retrieving Images and Searching Using Image Descriptors*. Now, in the current chapter, we will explore algorithms that have a good ability to generalize or extrapolate, in the sense that they can cope with the real-world diversity that exists within a given class of object. For example, different cars have different designs, and people can appear to be different shapes depending on the clothes they wear.Specifically, we will pursue the following objectives:

- Learning about another kind of feature descriptor: the **histogram of oriented gradients** (**HOG**) descriptor.
- Understanding **non-maximum suppression**, also called **non-maxima suppression** (**NMS**), which helps us choose the best of an overlapping set of detection windows.
- Gaining a high-level understanding of **support vector machines** (**SVMs**). These general-purpose classifiers are based on supervised machine learning, in a way that is similar to linear regression.
- Detecting people with a pre-trained classifier based on HOG descriptors.

- Training a **bag-of-words** (**BoW**) classifier to detect a car. For this sample, we will work with a custom implementation of an image pyramid, a sliding window, and NMS so that we can better understand the inner workings of these techniques.

Most of the techniques in this chapter are not mutually exclusive; rather, they work together as components of a detector. By the end of the chapter, you will know how to train and use classifiers that have practical applications on the streets!

Technical requirements

This chapter uses Python, OpenCV, and NumPy. Please refer back to *Chapter 1*, *Setting Up OpenCV*, for installation instructions.The completed code for this chapter can be found in this book's GitHub repository, at **https://github.com/PacktPublishing/Learning-OpenCV-5-Computer-Vision-with-Python-Fourth-Edition**, in the `chapter07` folder. Sample images can be found in the repository in the `images` folder.

Understanding HOG descriptors

HOG is a feature descriptor, so it belongs to the same family of algorithms as **scale-invariant feature transform** (**SIFT**), **speeded-up robust features** (**SURF**), and **Oriented FAST and rotated BRIEF** (**ORB**), which we covered in *Chapter 6*, *Retrieving Images and Searching Using Image Descriptors*. Like other feature descriptors, HOG is capable of delivering the type of information that is vital for feature matching, as well as for object detection and recognition. Most commonly, HOG is used for object detection. The algorithm – and, in particular, its use as a people detector – was popularized by Navneet Dalal and Bill Triggs in their paper *Histograms of Oriented Gradients for Human Detection* (INRIA, 2005), which is available online at **https://lear.inrialpes.fr/people/triggs/pubs/Dalal-cvpr05.pdf**.HOG's internal mechanism is really clever; an image is divided into cells and

a set of gradients is calculated for each cell. Each gradient describes the change in pixel intensities in a given direction. Together, these gradients form a histogram representation of the cell. We encountered a similar approach when we studied face recognition with the **local binary pattern histogram (LBPH)** in *Chapter 6*, *Detecting and Recognizing Faces*.Before diving into the technical details of how HOG works, let's first take a look at how HOG *sees* the world.

Visualizing HOG

Carl Vondrick, Aditya Khosla, Hamed Pirsiavash, Tomasz Malisiewicz, and Antonio Torralba have developed a HOG visualization technique called HOGgles (HOG goggles). For a summary of HOGgles, as well as links to code and publications, refer to Carl Vondrick's MIT web page at **http://www.cs.columbia.edu/~vondrick/ihog/index.html**. As one of their test images, Vondrick et al. use the following photograph of a truck:

Figure 7.1: A truck, to be used as a test image for HOG visualization

Vondrick et al. produce the following visualization of the HOG descriptors, based on an approach from Dalal and Triggs' earlier paper:

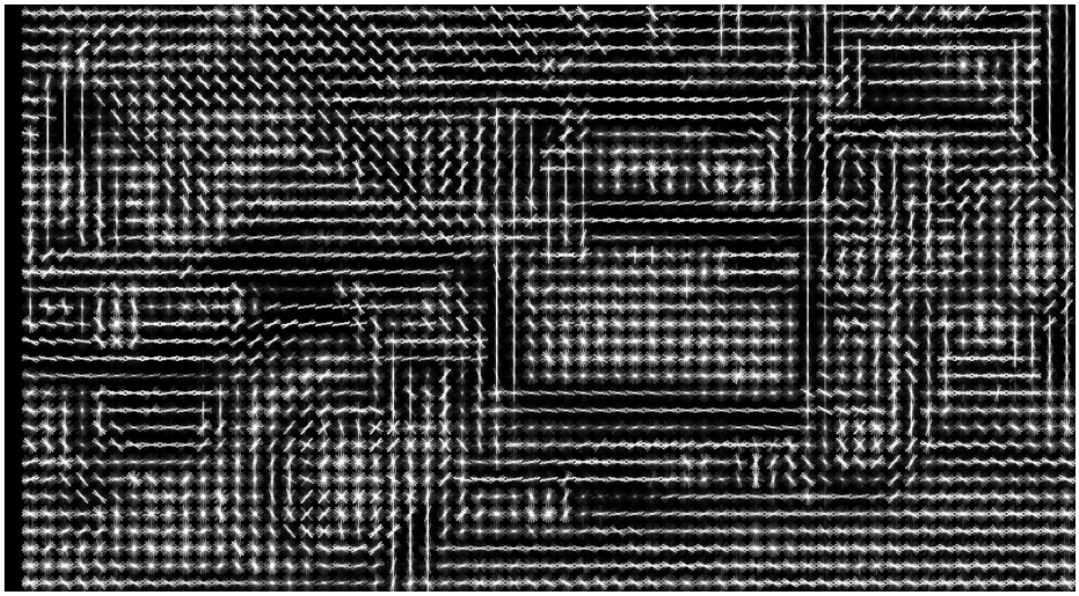

Figure 7.2: HOG cell gradients, visualized as lines

Then, applying HOGgles, Vondrick et al. invert the feature description algorithm to reconstruct the image of the truck as HOG sees it, as shown here:

Figure 7.3: HOG cell gradients, visualized as smooth grayscale transitions

In both of these two visualizations, you can see that HOG has divided the image into cells, and you can easily recognize the wheels and the

main structure of the vehicle. In the first visualization, the calculated gradients for each cell are visualized as a set of crisscrossing lines that sometimes look like an elongated star; the star's longer axes represent stronger gradients. In the second visualization, the gradients are visualized as a smooth transition in brightness, along various axes in a cell.Now, let's give further consideration to the way HOG works, and the way it can contribute to an object detection solution.

Using HOG to describe regions of an image

For each HOG cell, the histogram contains a number of bins equal to the number of gradients or, in other words, the number of axis directions that HOG considers. After calculating all the cells' histograms, HOG processes groups of histograms to produce higher-level descriptors. Specifically, the cells are grouped into larger regions, called blocks. These blocks can be made of any number of cells, but Dalal and Triggs found that 2x2 cell blocks yielded the best results when performing people detection. A block-wide vector is created so that it can be normalized, compensating for local variations in illumination and shadowing. (A single cell is too small a region to detect such variations.) This normalization improves a HOG-based detector's robustness, with respect to variations in lighting conditions.Like other detectors, a HOG-based detector needs to cope with variations in objects' location and scale. The need to search in various locations is addressed by moving a fixed-size sliding window across an image. The need to search at various scales is addressed by scaling the image to various sizes, forming a so-called image pyramid. We studied these techniques previously in *Chapter 5*, *Detecting and Recognizing Faces*, specifically in the *Conceptualizing Haar cascades* section. However, let's elaborate on one difficulty: how to handle multiple detections in overlapping windows.Suppose we are using a sliding window to perform people detection on an image. We slide our window in small steps, just a few pixels at a time, so we expect that it will frame any given person multiple times. Assuming that overlapping detections are indeed one person, we do not want to report multiple locations but, rather, only one

location that we believe to be correct. In other words, even if a detection at a given location has a *good* confidence score, we might reject it if an overlapping detection has a *better* confidence score; thus, from a set of overlapping detections, we would choose the one with the *best* confidence score.This is where NMS comes into play. Given a set of overlapping regions, we can suppress (or reject) all the regions for which our classifier did not produce a maximal score.

Understanding NMS

The concept of NMS might sound simple. From a set of overlapping solutions, just pick the best one! However, the implementation is more complex than you might initially think. Remember the image pyramid? Overlapping detections can occur at different scales. We must gather up all our positive detections, and convert their bounds back to a common scale before we check for overlap. A typical implementation of NMS takes the following approach:

1. Construct an image pyramid.
2. Scan each level of the pyramid with the sliding window approach, for object detection. For each window that yields a positive detection (beyond a certain arbitrary confidence threshold), convert the window back to the original image's scale. Add the window and its confidence score to a list of positive detections.
3. Sort the list of positive detections by order of descending confidence score so that the best detections come first in the list.
4. For each window, *W*, in the list of positive detections, remove all subsequent windows that significantly overlap with *W*. We are left with a list of positive detections that satisfy the criterion of NMS.

   Besides NMS, another way to filter the positive detections is to eliminate any subwindows. When we speak of a **subwindow** *(or subregion), we mean a window (or region in an image) that is entirely contained inside another window (or region). To check for subwindows, we simply need to compare the corner coordinates*

of various window rectangles. We will take this simple approach in our first practical example, in the Detecting people with HOG descriptors section. Optionally, NMS and suppression of subwindows can be combined.

Several of these steps are iterative, so we have an interesting optimization problem on our hands. A fast sample implementation in MATLAB is provided by Tomasz Malisiewicz at **http://www.computervisionblog.com/2011/08/blazing-fast-nmsm-from-exemplar-svm.html**. A port of this sample implementation to Python is provided by Adrian Rosebrock at **https://www.pyimagesearch.com/2015/02/16/faster-non-maximum-suppression-python/**. We will build atop the latter sample later in this chapter, in the *Detecting a car in a scene* section.Now, how do we determine the confidence score for a window? We need a classification system that determines whether a certain feature is present or not, and a confidence score for this classification. This is where **SVMs** come into play.

Understanding SVMs

Without going into details of how an SVM works, let's just try to grasp what it can help us accomplish in the context of machine learning and computer vision. Given labeled training data, an SVM learns to classify the same kind of data by finding an optimal *hyperplane*, which, in plain English, is the plane that divides differently labeled data by the largest possible margin. To aid our understanding, let's consider the following diagram, which is provided by Zach Weinberg under the Creative Commons Attribution-Share Alike 3.0 Unported License:

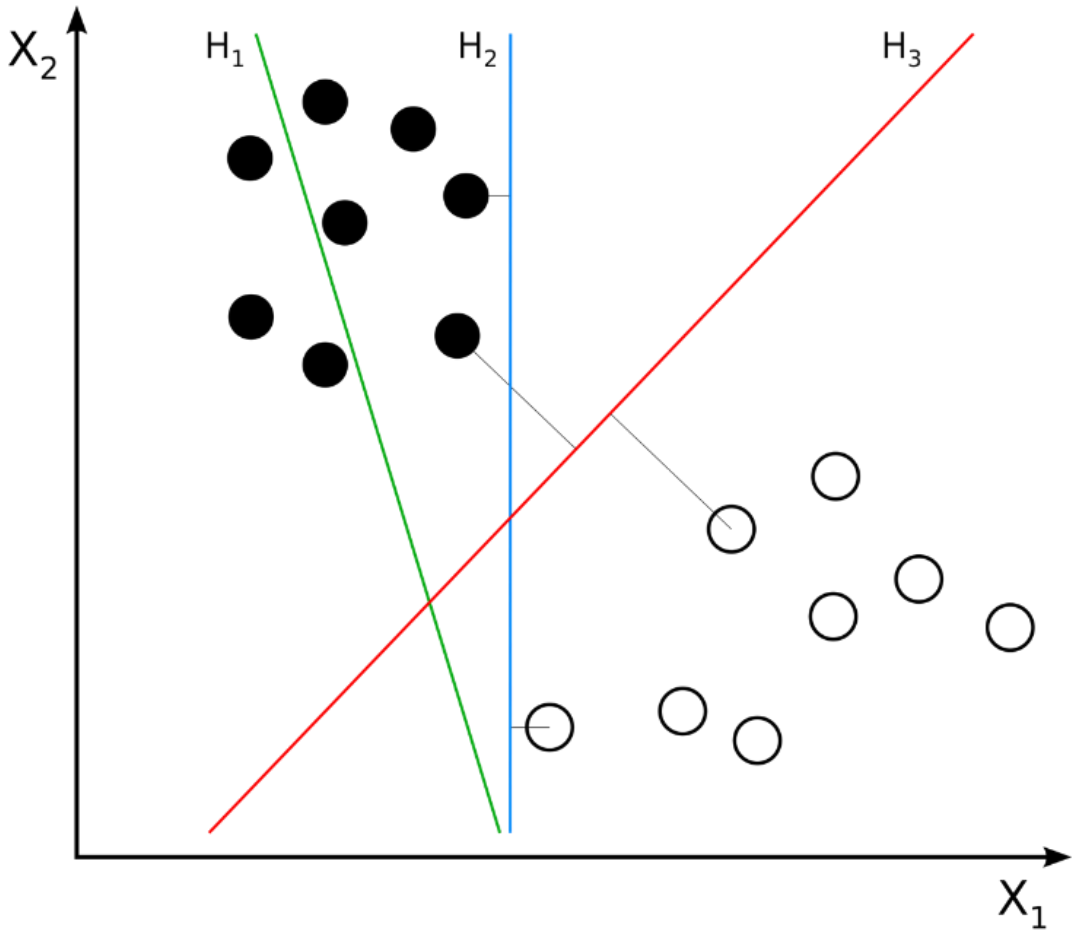

Figure 7.4: Clusters and hyperplanes. An SVM would identify **H3** *as the optimal hyperplane.*

Hyperplane **H1** (shown as a green line) does not divide the two classes (the black dots versus the white dots). Hyperplanes **H2** (shown as a blue line) and **H3** (shown as a red line) both divide the classes; however, only hyperplane **H3** divides the classes by a maximal margin.Let's suppose we are training an SVM as a people detector. We have two classes, *person* and *non-person*. As training samples, we provide vectors of HOG descriptors of various windows that do or do not contain a person. These windows may come from various images. The SVM learns by finding the optimal hyperplane that maximally divides the multidimensional HOG descriptor space into people (on one side of the hyperplane) and non-people (on the other side). Thereafter, when we give the trained SVM a vector of HOG descriptors for any other window in any image, the SVM can judge whether the window contains a person or not. The SVM can even give us a confidence value

that relates to the vector's distance from the optimal hyperplane.The SVM model has been around since the early 1960s. However, it has undergone improvements since then, and the basis of modern SVM implementations can be found in the paper *Support-vector networks* (*Machine Learning*, 1995) by Corinna Cortes and Vladimir Vapnik. It is available at **http://link.springer.com/article/10.1007/BF00994018**.Now that we have a conceptual understanding of the key components we can combine to make an object detector, we can start looking at a few examples. We will start with one of OpenCV's ready-made object detectors, and then we will progress to designing and training our own custom object detectors.

Detecting people with HOG descriptors

OpenCV comes with a class called `cv2.HOGDescriptor`, which is capable of performing people detection. The interface has some similarities to the `cv2.CascadeClassifier` class that we used in *Chapter 5*, *Detecting and Recognizing Faces*. However, unlike `cv2.CascadeClassifier`, `cv2.HOGDescriptor` sometimes returns nested detection rectangles. In other words, `cv2.HOGDescriptor` might tell us that it detected one person whose bounding rectangle is located completely inside another person's bounding rectangle. This situation really is possible; for example, a child could be standing in front of an adult, and the child's bounding rectangle could be completely inside the adult's bounding rectangle. However, in a typical situation, nested detections are probably errors, so `cv2.HOGDescriptor` is often used along with code to filter out any nested detections.Let's begin our sample script by implementing a test to determine whether one rectangle is nested inside another. For this purpose, we will write a function, `is_inside(i, o)`, where `i` is the possible inner rectangle and `o` is the possible outer rectangle. The function will return `True` if `i` is inside `o`; otherwise, it will return `False`. Here is the start of the script:

```
import cv2
def is_inside(i, o):
    ix, iy, iw, ih = i
    ox, oy, ow, oh = o
    return ix > ox and ix + iw < ox + ow and \
        iy > oy and iy + ih < oy + oh
```

Now, we create an instance of `cv2.HOGDescriptor`, and we specify that it will use a default people detector that is built into OpenCV by running the following code:

```
hog = cv2.HOGDescriptor()
hog.setSVMDetector(cv2.HOGDescriptor_getDefaultPeopleDetector())
```

Note that we specified the people detector with the `setSVMDetector` method. Hopefully, this makes sense based on the previous section of this chapter; an SVM is a classifier, so the choice of SVM determines the type of object our `cv2.HOGDescriptor` will detect.Now, we proceed to load an image – in this case, an old photograph of women working in a hayfield – and we attempt to detect people in the image by running the following code:

```
img = cv2.imread('../images/haying.jpg')
found_rects, found_weights = hog.detectMultiScale(
    img, winStride=(4, 4), scale=1.02, groupThreshold=1.9)
```

Note that `cv2.HOGDescriptor` has a `detectMultiScale` method, which returns two lists:

1. A list of bounding rectangles for detected objects (in this case, detected people).
2. A list of weights or confidence scores for detected objects. A higher value indicates greater confidence that the detection result is correct.

`detectMultiScale` accepts several optional arguments, including the following:

- `winStride`: This tuple defines the *x* and *y* distance that the sliding window moves between successive detection attempts. HOG works well with overlapping windows, so the stride may be small relative to the window size. A smaller value produces more detections, at a higher computational cost. The default stride has no overlap; it is the same as the window size, which is `(64, 128)` for the default people detector.
- `scale`: This scale factor is applied between successive levels of the image pyramid. A smaller value produces more detections, at a higher computational cost. The value must be greater than `1.0`. The default is `1.5`.
- `groupThreshold`: This value determines how stringent our detection criteria are. A smaller value is less stringent, resulting in more detections. The default is `2.0`.

Now, we can filter the detection results to remove nested rectangles. To determine whether a rectangle is a nested rectangle, we potentially need to compare it to every other rectangle. Note the use of our `is_inside` function in the following nested loop:

```
found_rects_filtered = []
found_weights_filtered = []
for ri, r in enumerate(found_rects):
    for qi, q in enumerate(found_rects):
        if ri != qi and is_inside(r, q):
            break
    else:
        found_rects_filtered.append(r)
        found_weights_filtered.append(found_weights[ri])
```

Finally, let's draw the remaining rectangles and weights in order to highlight the detected people, and let's show and save this visualization, as follows:

```
for ri, r in enumerate(found_rects_filtered):
    x, y, w, h = r
    cv2.rectangle(img, (x, y), (x + w, y + h), (0, 255, 255), 2)
    text = '%.2f' % found_weights_filtered[ri]
    cv2.putText(img, text, (x, y - 20),
                cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 255), 2)
cv2.imshow('Women in Hayfield Detected', img)
cv2.imwrite('./women_in_hayfield_detected.jpg', img)
cv2.waitKey(0)
```

If you run the script yourself, you will see rectangles around people in the image. Here is the result:

Figure 7.5: People detected using HOG. The photograph is another example of the work of Sergey Prokudin-Gorsky (1863-1944), a pioneer of color photography. Here, the scene is a field at the Leushinskii Monastery, in northwestern Russia, in 1909.

Of the six women who are nearest to the camera, five have been successfully detected. At the same time, a tower in the background has been falsely detected as a person. In many real-world applications, people-detection results can be improved by analyzing a series of frames in a video. For example, imagine that we are looking at a surveillance video of the Leushinskii Monastery's hayfield, instead of a single photograph. We should be able to add code to determine that the tower cannot be a person because it does not move. Also, we should be able to detect additional people in other frames and track each person's movement from frame to frame. We will look at a people-tracking problem in *Chapter 8*, *Tracking Objects*.Meanwhile, let's proceed to look at another kind of detector that we can train to detect a given class of objects.

Creating and training an object detector

Using a pre-trained detector makes it easy to build a quick prototype, and we are all very grateful to the OpenCV developers for making such useful capabilities as face detection and people detection readily available. However, whether you are a hobbyist or a computer vision professional, it is unlikely that you will only deal with people and faces.Moreover, if you are like the authors of this book, you will wonder how the people detector was created in the first place and whether you can improve it. Furthermore, you may also wonder whether you can apply the same concepts to detect diverse objects, ranging from cars to goblins.Indeed, in industry, you may have to deal with problems of detecting very specific objects, such as registration plates, book covers, or whatever thing may be most important to your employer or client.Thus, the question is, how do we come up with our own classifiers?There are many popular approaches. Throughout the remainder of this chapter, we will see that one answer lies in SVMs and the BoW technique.We have already talked about SVMs and HOG. Let's now take a closer look at BoW.

Understanding BoW

BoW is a concept that was not initially intended for computer vision; rather, we use an evolved version of this concept in the context of computer vision. Let's first talk about its basic version, which – as you may have guessed – originally belongs to the field of language analysis and information retrieval.

> *Sometimes, in the context of computer vision, BoW is called* **bag of visual words** *(***BoVW)***. *However, we will simply use the term BoW, since this is the term used by OpenCV.*

BoW is the technique by which we assign a weight or count to each word in a series of documents; we then represent these documents with vectors of these counts. Let's look at an example, as follows:

- **Document 1**: I like OpenCV and I like Python.
- **Document 2**: I like C++ and Python.
- **Document 3**: I don't like artichokes.

These three documents allow us to build a dictionary – also called a **codebook** or vocabulary – with these values, as follows:

```
{
    I: 4,
    like: 4,
    OpenCV: 1,
    and: 2,
    Python: 2,
    C++: 1,
    don't: 1,
    artichokes: 1
}
```

We have eight entries. Let's now represent the original documents using eight-entry vectors. Each vector contains values representing the counts of all words in the dictionary, in order, for a given document.

The vector representation of the preceding three sentences is as follows:

```
[2, 2, 1, 1, 1, 0, 0, 0]
[1, 1, 0, 1, 1, 1, 0, 0]
[1, 1, 0, 0, 0, 0, 1, 1]
```

These vectors can be conceptualized as a histogram representation of documents or as a descriptor vector that can be used to train classifiers. For example, a document can be classified as *spam* or *not spam* based on such a representation. Indeed, spam filtering is one of the many real-world applications of BoW.Now that we have a grasp of the basic concept of BoW, let's see how this applies to the world of computer vision.

Applying BoW to computer vision

We are by now familiar with the concepts of features and descriptors. We have used algorithms such as SIFT and SURF to extract descriptors from an image's features so that we can match these features in another image.We have also recently familiarized ourselves with another kind of descriptor, based on a codebook or dictionary. We know about an SVM, a model that can accept labeled descriptor vectors as training data, can find an optimal division of the descriptor space into the given classes, and can predict the classes of new data.Armed with this knowledge, we can take the following approach to build a classifier:

1. Take a sample dataset of images.
2. For each image in the dataset, extract descriptors (with SIFT, SURF, ORB, or a similar algorithm).
3. Add each descriptor vector to the BoW trainer.
4. Cluster the descriptors into *k* clusters whose centers (centroids) are our visual words. This last point probably sounds a bit obscure, but we will explore it further in the next section.

At the end of this process, we have a dictionary of visual words ready to be used. As you can imagine, a large dataset will help make our dictionary richer in visual words. Up to a point, the more words, the better!Having trained a classifier, we should proceed to test it. The good news is that the test process is conceptually very similar to the training process outlined previously. Given a test image, we can extract descriptors and **quantize** them (or reduce their dimensionality) by calculating a histogram of their distances to the centroids. Based on this, we can attempt to recognize visual words, and locate them in the image.This is the point in the chapter where you have built up an appetite for a deeper practical example, and are raring to code.
However, before proceeding, let's take a quick but necessary digression into the theory of *k*-means clustering so that you can fully understand how visual words are created. Thereby, you will gain a better understanding of the process of object detection using BoW and SVMs.

k-means clustering

k-means clustering is a method of quantization whereby we analyze a large number of vectors in order to find a small number of clusters. Given a dataset, *k* represents the number of clusters into which the dataset is going to be divided. The term *means* refers to the mathematical concept of the mean or the average; when visually represented, the mean of a cluster is its **centroid** or the geometric center of points in the cluster.

> **Clustering** *refers to the process of grouping points in a dataset into clusters.*

OpenCV provides a class called `cv2.BOWKMeansTrainer`, which we will use to help train our classifier. As you might expect, the OpenCV documentation gives the following summary of this class:

> *A "kmeans-based class to train a visual vocabulary using the bag of words approach."*

After this long theoretical introduction, we can look at an example, and start training our custom classifier.

Detecting cars

To train any kind of classifier, we must begin by creating or acquiring a training dataset. We are going to train a car detector, so our dataset must contain positive samples that represent cars, as well as negative samples that represent other (non-car) things that the detector is likely to encounter while looking for cars. For example, if the detector is intended to search for cars on a street, then a picture of a curb, a crosswalk, a pedestrian, or a bicycle might be a more representative negative sample than a picture of the rings of Saturn. Besides representing the expected subject matter, ideally, the training samples should represent the way our particular camera and algorithm will see the subject matter.Ultimately, in this chapter, we intend to use a sliding window of fixed size, so it is important that our training samples conform to a fixed size, and that the positive samples are tightly cropped in order to frame a car without much background.Up to a point, we expect that the classifier's accuracy will improve as we keep adding good training images. On the other hand, a large dataset can make the training slow, and it is possible to overtrain a classifier such that it fails to extrapolate beyond the training set. Later in this section, we will write our code in a way that allows us to easily modify the number of training images so that we find a good size experimentally.Assembling a dataset of car images would be a time-consuming task if we were to do it all by ourselves (though it is entirely doable). To avoid reinventing the wheel – or the whole car – we can avail ourselves of ready-made datasets, such as the following:

- **UIUC Image Database for Car Detection:** **https://cogcomp.seas.upenn.edu/Data/Car/**
- **Stanford Cars Dataset**: **http://ai.stanford.edu/~jkrause/cars/car_dataset.html**

Let's use the UIUC dataset in our example. Several steps are involved in obtaining this dataset and using it in a script, so let's walk through them one by one, as follows:

1. Download the UIUC dataset from **https://github.com/gcr/arc-evaluator/raw/master/CarData.tar.gz**. Unzip it to some folder, which we will refer to as `<project_path>`. Now, the unzipped data should be located at `<project_path>/CarData`. Specifically, we will use some of the images in `<project_path>/CarData/TrainImages` and `<project_path>/CarData/TestImages`.
2. Also in `<project_path>`, let's create a Python script called `detect_car_bow_svm.py`. To begin the script's implementation, write the following code to check whether the `CarData` subfolder exists:

```
import cv2
import numpy as np
import os
if not os.path.isdir('CarData'):
    print(
        'CarData folder not found. Please download and unzip '
        'https://github.com/gcr/arc-evaluator/raw/master/CarData.tar.gz '
        'into the same folder as this script.')
    exit(1)
```

If you can run this script and it does not print anything, this means everything is in its correct place.

1. Next, let's define the following constants in the script:

```
BOW_NUM_TRAINING_SAMPLES_PER_CLASS = 10
SVM_NUM_TRAINING_SAMPLES_PER_CLASS = 11
BOW_NUM_CLUSTERS = 40
```

Note that our classifier will make use of two training stages: one stage for the BoW vocabulary, which will use a number of images as samples, and another stage for the SVM, which will use a number of BoW descriptor vectors as samples. Arbitrarily, we have defined a different number of training samples for each stage. At each stage, we could have also defined a different number of training samples for the two classes (*car* and *not car*), but instead, we will use the same number. We have also defined the number of BoW clusters. Note that the number of BoW clusters may be larger than the number of BoW training images, since each image has many descriptors.

1. We will use `cv2.SIFT` to extract descriptors and `cv2.FlannBasedMatcher` to match these descriptors. Let's initialize these algorithms with the following code:

```
sift = cv2SIFT_create()
FLANN_INDEX_KDTREE = 1
index_params = dict(algorithm=FLANN_INDEX_KDTREE, trees=5)
search_params = dict(checks=50))
flann = cv2.FlannBasedMatcher(index_params, search_params)
```

Note that we have initialized SIFT and the **Fast Library for Appropriate Nearest Neighbors** (**FLANN**) in the same way as we did in *Chapter 6, Retrieving Images and Searching Using image Descriptors*. However, this time, descriptor matching is not our end goal; instead, it will be part of the functionality of our BoW.

1. OpenCV provides a class called `cv2.BOWKMeansTrainer` to train a BoW vocabulary, and a class called `cv2.BOWImgDescriptorExtractor` to convert some kind of lower-level descriptors – in our example, SIFT descriptors – into BoW descriptors. Let's initialize these objects with the following code:

```
bow_kmeans_trainer = cv2.BOWKMeansTrainer(BOW_NUM_CLUSTERSCLUSTERS)
bow_extractor = cv2.BOWImgDescriptorExtractor(sift, flann)
```

When initializing `cv2.BOWKMeansTrainer`, we must specify the number of clusters – in our example, `40`, as defined earlier. When initializing `cv2.BOWImgDescriptorExtractor`, we must specify a descriptor extractor and a descriptor matcher – in our example, the `cv2.SIFT` and `cv2.FlannBasedMatcher` objects that we created earlier.

1. To train the BoW vocabulary, we will provide samples of SIFT descriptors for various *car* and *not car* images. We will load the images from the `CarData/TrainImages` subfolder, which contains positive (*car*) images with names such as `pos-x.pgm`, and negative (*not car*) images with names such as `neg-x.pgm`, where `x` is a number starting at `1`. Let's write the following utility function to return a pair of paths to the `i`th positive and negative training images, where `i` is a number starting at `0`:

```
def get_pos_and_neg_paths(i):
    pos_path = 'CarData/TrainImages/pos-%d.pgm' % (i+1)
    neg_path = 'CarData/TrainImages/neg-%d.pgm' % (i+1)
    return pos_path, neg_path
```

Later in this section, we will call the preceding function in a loop, with a varying value of `i`, when we need to acquire a number of training samples.

1. For each path to a training sample, we will need to load the image, extract SIFT descriptors, and add the descriptors to the BoW vocabulary trainer. Let's write another utility function to do precisely this, as follows:

```
def add_sample(path):
    img = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
    keypoints, descriptors = sift.detectAndCompute(img, None)
    if descriptors is not None:
        bow_kmeans_trainer.add(descriptors)
```

If no features are found in the image, then the `keypoints` and descriptors variables will be `None`.

1. At this stage, we have everything we need to start training the BoW vocabulary. Let's read a number of images for each class (*car* as the positive class and *not car* as the negative class) and add them to the training set, as follows:

```
for i in range(BOW_NUM_TRAINING_SAMPLES_PER_CLASS):
    pos_path, neg_path = get_pos_and_neg_paths(i)
    add_sample(pos_path)
    add_sample(neg_path)
```

1. Now that we have assembled the training set, we will call the vocabulary trainer's `cluster` method, which performs the *k*-means classification and returns the vocabulary. We will assign this vocabulary to the BoW descriptor extractor, as follows:

```
voc = bow_kmeans_trainer.cluster()
bow_extractor.setVocabulary(voc)
```

Remember that earlier, we initialized the BoW descriptor extractor with a SIFT descriptor extractor and FLANN matcher. Now, we have also given the BoW descriptor extractor a vocabulary that we trained with samples of SIFT descriptors. At this stage, our BoW descriptor extractor has everything it needs in order to extract BoW descriptors from **Difference of Gaussian** (**DoG**) features.

Remember that `cv2.SIFT` detects DoG features and extracts SIFT descriptors, as we discussed in Chapter 6, Retrieving Images and Searching Using Image Descriptors, specifically in the Detecting DoG features and extracting SIFT descriptors section.

1. Next, we will declare another utility function that takes an image and returns the descriptor vector, as computed by the BoW descrip-

tor extractor. This involves extracting the image's DoG features, and computing the BoW descriptor vector based on the DoG features, as follows:

```
def extract_bow_descriptors(img):
    features = sift.detect(img)
    return bow_extractor.compute(img, features)
```

1. We are ready to assemble another kind of training set, containing samples of BoW descriptors. Let's create two arrays to accommodate the training data and labels, and populate them with the descriptors generated by our BoW descriptor extractor. We will label each descriptor vector with `1` for a positive sample and `-1` for a negative sample, as shown in the following code block:

```
training_data = []
training_labels = []
for i in range(SVM_NUM_TRAINING_SAMPLES_PER_CLASS):
    pos_path, neg_path = get_pos_and_neg_paths(i)
    pos_img = cv2.imread(pos_path, cv2.IMREAD_GRAYSCALE)
    pos_descriptors = extract_bow_descriptors(pos_img)
    if pos_descriptors is not None:
        training_data.extend(pos_descriptors)
        training_labels.append(1)
    neg_img = cv2.imread(neg_path, cv2.IMREAD_GRAYSCALE)
    neg_descriptors = extract_bow_descriptors(neg_img)
    if neg_descriptors is not None:
        training_data.extend(neg_descriptors)
        training_labels.append(-1)
```

Should you wish to train a classifier to distinguish between multiple positive classes, you can simply add other descriptors with other labels. For example, we could train a classifier that uses the label 1 for car, 2 for person, and -1 for background. There is no requirement to have a negative or background class but, if you do

not, your classifier will assume that everything belongs to one of the positive classes.

1. OpenCV provides a class called `cv2.ml_SVM`, representing an SVM. Let's create an SVM, and train it with the data and labels that we previously assembled, as follows:

```
svm = cv2.ml.SVM_create()
svm.train(np.array(training_data), cv2.ml.ROW_SAMPLE,
          np.array(training_labels))
```

Note that we must convert the training data and labels from lists to NumPy arrays before we pass them to the `train` method of `cv2.ml_SVM`.

1. Finally, we are ready to test the SVM by classifying some images that were not part of the training set. We will iterate over a list of paths to test images. For each path, we will load the image, extract BoW descriptors, and get the SVM's **prediction** or classification result, which will be either 1.0 (*car*) or -1.0 (*not car*), based on the training labels we used earlier. We will draw text on the image to show the classification result, and we will show the image in a window. After showing all the images, we will wait for the user to hit any key, and then the script will end. All of this is achieved in the following block of code:

```
for test_img_path in ['CarData/TestImages/test-0.pgm',
                      'CarData/TestImages/test-1.pgm',
                      '../images/car.jpg',
                      '../images/haying.jpg',
                      '../images/statue.jpg',
                      '../images/woodcutters.jpg']:
    img = cv2.imread(test_img_path)
    gray_img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    descriptors = extract_bow_descriptors(gray_img)
    prediction = svm.predict(descriptors)
```

```
    if prediction[1][0][0] == 1.0:
        text = 'car'
        color = (0, 255, 0)
    else:
        text = 'not car'
        color = (0, 0, 255)
    cv2.putText(img, text, (10, 30),
cv2.FONT_HERSHEY_SIMPLEX, 1,
                color, 2, cv2.LINE_AA)
    cv2.imshow(test_img_path, img)
cv2.waitKey(0)
```

Save and run the script. You should see six windows with various classification results. Here is a screenshot of one of the true positive results:

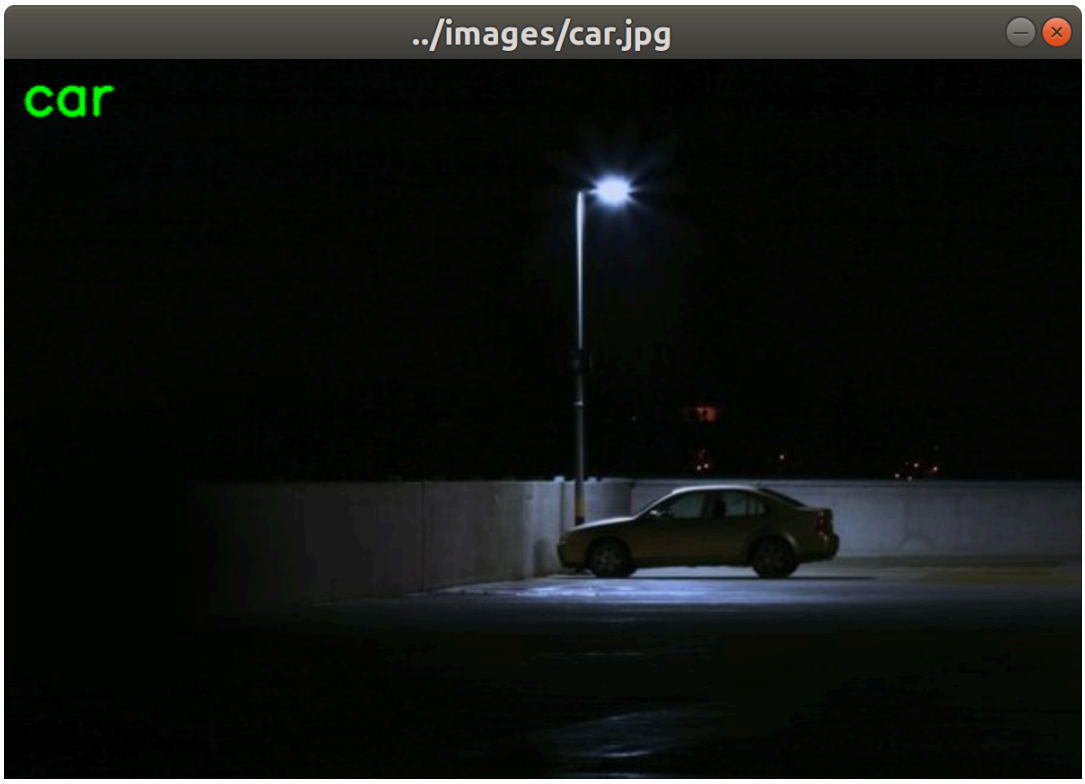

Figure 7.6: An image of a car, correctly classified by our SVM

The next screenshot shows one of the true negative results:

Figure 7.7: A non-car image, correctly classified by our SVM

Of the six images in our simple test, only the following one is incorrectly classified:

Figure 7.8: A non-car image, incorrectly classified as a car by our SVM

Incorrect classification solely depends upon the amount and nature of positive and negative images used for training and the robustness of the learning algorithm. For example, if the car position in every image of our training dataset is in the center of the image, the test image with a car position at the corner of the image may classify incorrectly. Experiment with adjusting the number of training samples and other parameters, and try testing the classifier on more images, to see what results you can get. Let's take stock of what we have done so far. We have used a mixture of SIFT, BoW, and SVMs to train a classifier to distinguish between two classes: *car* and *not car*. We have applied this classifier to whole images. The next logical step is to apply a sliding window technique so that we can narrow down our classification results to specific regions of an image.

Combining an SVM with a sliding window

By combining our SVM classifier with a sliding window technique and an image pyramid, we can achieve the following improvements:

- Detect multiple objects of the same kind in an image.
- Determine the position and size of each detected object in an image.

We will adopt the following approach:

1. Take a region of the image, classify it, and then move this window to the right by a predefined step size. When we reach the rightmost end of the image, reset the *x* coordinate to **0**, move down a step, and repeat the entire process.
2. At each step, perform a classification with the SVM that was trained with BoW.
3. Keep track of all the windows that are positive detections, according to the SVM.
4. After classifying every window in the entire image, scale the image down, and repeat the entire process of using a sliding window. Thus, we are using an image pyramid. Continue rescaling and classifying until we get to a minimum size.

When we reach the end of this process, we have collected important information about the content of the image. However, there is a problem: in all likelihood, we have found a number of overlapping blocks that each yield a positive detection with high confidence. That is to say, the image may contain one object that gets detected multiple times. If we reported these multiple detections, our report would be quite misleading, so we will filter our results using NMS.

> *For a refresher, you may wish to refer back to the Understanding NMS section, earlier in this chapter.*

Next, let's look at how to modify and extend our previous script, in order to implement the approach we have just described.

Detecting a car in a scene

We are now ready to apply all the concepts we learned so far by creating a car detection script that scans an image and draws rectangles around cars. Let's create a new Python script, `detect_car_bow_svm_sliding_window.py`, by copying our previous script, `detect_car_bow_svm.py`. (We covered the implementation of `detect_car_bow_svm.py` earlier, in the *Detecting cars* section.) Much of the new script's implementation will remain unchanged because we still want to train a BoW descriptor extractor and an SVM in almost the same way as we did previously. However, after the training is complete, we will process the test images in a new way. Rather than classifying each image in its entirety, we will decompose each image into pyramid layers and windows, we will classify each window, and we will apply NMS to a list of windows that yielded positive detections.For NMS, we will rely on Malisiewicz and Rosebrock's implementation, as described earlier in this chapter, in the *Understanding NMS* section. You can find a slightly modified copy of their implementation in this book's GitHub repository, specifically in the Python script at `chapter07/non_max_suppression.py`. This script provides a function with the following signature:

```
def non_max_suppression_fast(boxes, overlapThresh):
```

As its first argument, the function takes a NumPy array containing rectangle coordinates and scores. If we have *N* rectangles, the shape of this array is *N*x5. For a given rectangle at index *i*, the values in the array have the following meanings:

- `boxes[i][0]` is the leftmost *x* coordinate.
- `boxes[i][1]` is the topmost *y* coordinate.
- `boxes[i][2]` is the rightmost *x* coordinate.

- `boxes[i][3]` is the bottommost *y* coordinate.
- `boxes[i][4]` is the score, where a higher score represents greater confidence that the rectangle is a correct detection result.

As its second argument, the function takes a threshold that represents the maximum proportion of overlap between rectangles. If two rectangles have a greater proportion of overlap than this, the one with the lower score will be filtered out. Ultimately, the function will return an array of the remaining rectangles.Now, let's turn our attention to the modifications to the `detect_car_bow_svm_sliding_window.py` script, as follows:

1. First, we want to add a new import statement for the NMS function, as `highlighted` in the following code:

```
import cv2
import numpy as np
import os
from non_max_suppression import non_max_suppression_fast as nms
```

1. Let's define some additional parameters near the start of the script, as `highlighted` here:

```
BOW_NUM_TRAINING_SAMPLES_PER_CLASS = 10
SVM_NUM_TRAINING_SAMPLES_PER_CLASS = 11
BOW_NUM_CLUSERS = 12
SVM_SCORE_THRESHOLD = 2.22
NMS_OVERLAP_THRESHOLD = 0.44
```

Note that we have reduced the number of *k*-means clusters from `40` to `12` (a number chosen arbitrarily based on experimentation). We will use `SVM_SCORE_THRESHOLD` as a threshold to distinguish between a positive window and a negative window. We will see how the score is obtained a little later in this section. We will use `NMS_OVERLAP_THRESHOLD` as the maximum acceptable proportion of

overlap in the NMS step. Here, we have arbitrarily chosen 15%, so we will cull windows that overlap by more than this proportion. As you experiment with your SVMs, you may tweak these parameters to your liking until you find values that yield the best results in your application.

1. We will also adjust the parameters of the SVM, as follows:

```
svm = cv2.ml.SVM_create()
svm.setType(cv2.ml.SVM_C_SVC)
svm.setC(50)
svm.train(np.array(training_data), cv2.ml.ROW_SAMPLE,
          np.array(training_labels))
```

With the preceding changes to the SVM, we are specifying the classifier's level of strictness or severity. As the value of the `C` parameter increases, the risk of false positives decreases but the risk of false negatives increases. In our application, a false positive would be a window detected as a car when it is really *not* a car, and a false negative would be a car detected as a window when it really *is* a car.After the code that trains the SVM, we want to add two more helper functions. One of them will generate levels of the image pyramid, and the other will generate regions of interest, based on the sliding window technique. Besides adding these helper functions, we also need to handle the test images differently in order to make use of the sliding window and NMS. The following steps cover the changes:

1. First, let's look at the helper function that deals with the image pyramid. This function is shown in the following code block:

```
def pyramid(img, scale_factor=1.005, min_size=(10040100, 40),
            max_size=(600, 240240)):
    h, w = img.shape
    min_w, min_h = min_size
    max_w, max_h = max_size
    while w >= min_w and h >= min_h:
```

```
        if w <= max_w and h <= max_h:
            yield img
        w /= scale_factor
        h /= scale_factor
        img = cv2.resize(img, (int(w), int(h)),
                         interpolation=cv2.INTER_AREA)
```

The preceding function takes an image and generates a series of resized versions of it. The series is bounded by a maximum and minimum image size.

> *You will have noticed that the resized image is not returned with the `return` keyword but with the `yield` keyword. This is because this function is a so-called generator. It produces a series of images that we can easily use in a loop. If you are not familiar with generators, take a look at the official Python Wiki at* **https://wiki.python.org/moin/Generators**.

1. Next up is the function to generate regions of interest, based on the sliding window technique. This function is shown in the following code block:

```
def sliding_window(img, step=20, window_size=(100, 40)):
    img_h, img_w = img.shape
    window_w, window_h = window_size
    for y in range(0, img_w, step):
        for x in range(0, img_h, step):
            roi = img[y:y+window_h, x:x+window_w]
            roi_h, roi_w = roi.shape
            if roi_w == window_w and roi_h == window_h:
                yield (x, y, roi)
```

Again, this is a generator. Although it is a bit deep-nested, the mechanism is very simple: given an image, return the upper-left coordinates and the sub-image representing the next window. Successive windows are shifted by an arbitrarily sized step from left to right until we reach

the end of a row, and from the top to bottom until we reach the end of the image.

1. Now, let's consider the treatment of test images. As in the previous version of the script, we loop through a list of paths to test images, in order to load and process each one. The beginning of the loop is unchanged. For context, here it is:

```
for test_img_path in ['CarData/TestImages/test-0.pgm',
                      'CarData/TestImages/test-1.pgm',
                      '../images/car.jpg',
                      '../images/haying.jpg',
                      '../images/statue.jpg',
                      '../images/woodcutters.jpg']:
    img = cv2.imread(test_img_path)
    gray_img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

For each test image, we iterate over the pyramid levels, and for each pyramid level, we iterate over the sliding window positions. For each window or **region of interest** (**ROI**), we extract BoW descriptors and classify them using the SVM. If the classification produces a positive result that passes a certain confidence threshold, we add the rectangle's corner coordinates and confidence score to a list of positive detections. Continuing from the previous code block, we proceed to handle a given test image with the following code:

```
    pos_rects = []
    for resized in pyramid(gray_img):
        for x, y, roi in sliding_window(resized):
            descriptors = extract_bow_descriptors(roi)
            if descriptors is None:
                continue
            prediction = svm.predict(descriptors)
            if prediction[1][0][0] == 1.0:
                raw_prediction = svm.predict(
                    descriptors,
                    flags=cv2.ml.STAT_MODEL_RAW_OUTPUT)
```

```
                score = -raw_prediction[1][0][0]
                if score > SVM_SCORE_THRESHOLD:
                    h, w = roi.shape
                    scale = gray_img.shape[0] / \
                        float(resized.shape[0])
                    pos_rects.append([int(x * scale),
                                      int(y * scale),
                                      int((x+w) * scale),
                                      int((y+h) * scale),
                                      score])
```

Let's take note of a couple of complexities in the preceding code, as follows:

- To obtain a confidence score for the SVM's prediction, we must run the `predict` method with an optional flag, `cv2.ml.STAT_MODEL_RAW_OUTPUT`. Then, instead of returning a label, the method returns a score as part of its output. This score may be negative, and a low value represents a high level of confidence. To make the score more intuitive – and to match the NMS function's assumption that a higher score is better – we negate the score so that a high value represents a high level of confidence.
- Since we are working with multiple pyramid levels, the window coordinates do not have a common scale. We have converted them back to a common scale – the original image's scale – before adding them to our list of positive detections.

So far, we have performed car detection at various scales and positions; as a result, we have a list of detected car rectangles, including coordinates and scores. We expect a lot of overlap within this list of rectangles.

1. Now, let's call the NMS function, in order to cherry-pick the highest-scoring rectangles in the case of overlap, as follows:

```
    pos_rects = nms(np.array(pos_rects), NMS_OVERLAP_THRESHOLD)
```

Note that we have converted our list of rectangle coordinates and scores to a NumPy array, which is the format expected by this function.

At this stage, we have an array of detected car rectangles and their scores, and we have ensured that these are the best non-overlapping detections we can select (within the parameters of our model).

1. Now, let's draw the rectangles and their scores by adding the following inner loop to the code:

```
    for x0, y0, x1, y1, score in pos_rects:
        cv2.rectangle(img, (int(x0), int(y0)), (int(x1), int(y1)),
                      (0, 255, 255), 2)
        text = '%.2f' % score
        cv2.putText(img, text, (int(x0), int(y0) - 20),
                    cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 255), 2)
```

As in the previous version of this script, the body of the outer loop ends by showing the current test image, including the annotations we have drawn on it. After the loop runs through all the test images, we wait for the user to press any key; then, the program ends, as shown here:

```
    cv2.imshow(test_img_path, img)
cv2.waitKey(0)
```

Let's run the modified script, and see how well it can answer the eternal question: *Dude, where's my car?*The following screenshot shows a pair of successful detections:

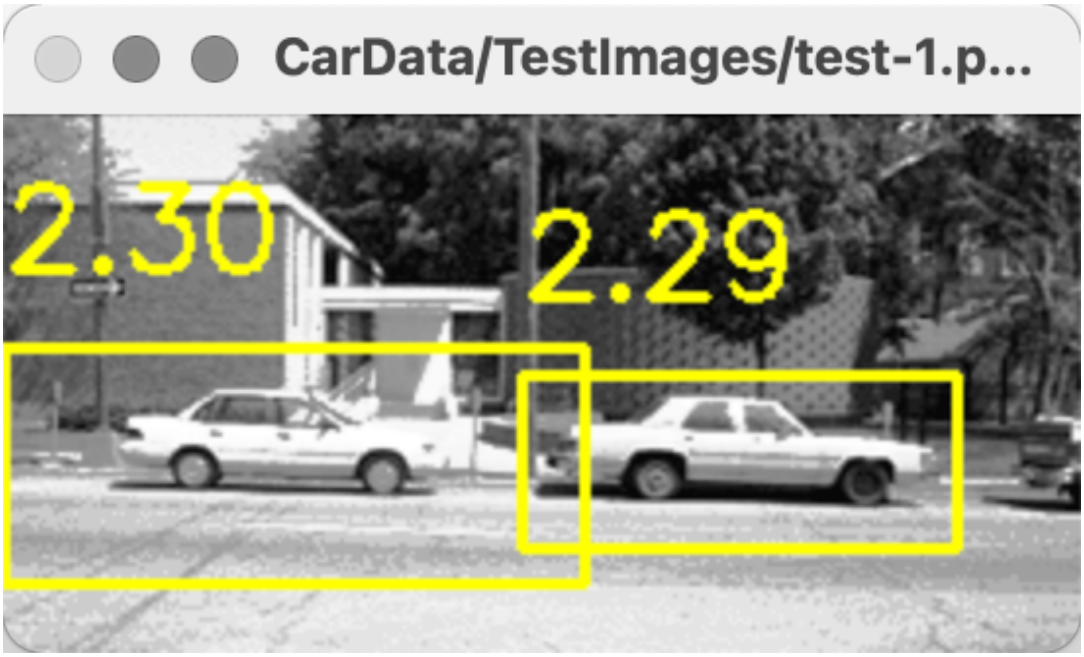

Figure 7.9: A pair of cars, correctly detected by our SVM with a sliding window approach

Among our other 5 test images, we have 2 false negatives (where a car is not detected as such) but no false positives.

> *Remember that in this sample script, our training sets are small. Larger training sets, with more diverse backgrounds, could improve the results. Also, remember that the image pyramid and sliding window are producing a large number of ROIs. When we consider this, we should realize that aa low false positive rate is actually a significant accomplishment. Moreover, if we were performing detection on frames of a video, we could mitigate the problem of false negatives because we would have a chance to detect a car in multiple frames.*

Feel free to experiment with the parameters and training sets of the preceding script. When you are ready, let's wrap up this chapter with a few closing notes.

Saving and loading a trained SVM

A final piece of advice on SVMs: you do not need to train a detector every time you want to use it – and, indeed, you should avoid doing so because training is slow. You can use code such as the following to save a trained SVM model to an XML file:

```
svm = cv2.ml.SVM_create()
svm.train(np.array(training_data), cv2.ml.ROW_SAMPLE,
          np.array(training_labels))
svm.save('my_svm.xml')
```

Subsequently, you can reload the trained SVM, using code such as the following:

```
svm = cv2.ml.SVM_create()
svm.load('my_svm.xml')
```

Typically, you might have one script that trains and saves your SVM model, and other scripts that load and use it for various detection problems.

Summary

In this chapter, we covered a wide range of concepts and techniques, including HOG, BoW, SVMs, image pyramids, sliding windows, and NMS. We learned that these techniques have applications in object detection, as well as other fields. We wrote a script that combined most of these techniques – BoW, SVMs, an image pyramid, a sliding window, and NMS – and we gained practical experience in machine learning through the exercise of training and testing a custom detector. Finally, we demonstrated that we can detect cars!Our new knowledge forms the foundation of the next chapter, in which we will utilize object detection and classification techniques on sequences of frames in videos. We will learn how to track objects and retain information about them – an important objective in many real-world applications.

Join our book community on Discord

https://discord.gg/djGjeMECxw